

Solved Exam – OOP 2022

Answer Question 1

1. The only technical difference between structures and classes in C++ is that **members of a structure are public by default, while members of a class are private by default.**
2. Member functions defined inside a class definition are **inline** by default.
3. In the UML, inheritance is called **generalization**.
4. **Virtual functions** enable polymorphic programming as an alternative to switch logic programming.
5. A constructor is executed automatically when an object is **created**.
6. To be accessed from a member function of the derived class, data or functions in the base class must be public or **protected**.
7. To call a function appropriately at execution time is known as **dynamic binding** (or **late binding**).
8. If a class contains at least one pure virtual function, it's an **abstract** class.
9. Constructors cannot be declared **virtual**.
10. **A friend function** is a function that is used to operate on objects of two different classes as arguments, and operate on their private data.

Answer Question 2

- Stack operations using multiple exception handlers.

```
#include <iostream>
using namespace std;

const int MAX = 3;

class Stack
{
private:
    int st[MAX];
    int top;
public:
    class Full { };
    class Empty { };

    Stack() : top(-1) { }

    void push(int var)
    {
        if(top >= MAX-1)
            throw Full();
        st[++top] = var;
    }

    int pop()
    {
        if(top < 0)
            throw Empty();
        return st[top--];
    }
};

int main()
{
    Stack s1;

    try {
        s1.push(11);
        s1.push(22);
        s1.push(33);

        cout << "1: " << s1.pop() << endl;
        cout << "2: " << s1.pop() << endl;
        cout << "3: " << s1.pop() << endl;
        cout << "4: " << s1.pop() << endl;
    }
    catch(Stack::Full)
    {
        cout << "Exception: Stack Full" << endl;
    }
    catch(Stack::Empty)
    {
        cout << "Exception: Stack Empty" << endl;
    }

    return 0;
}
```

Output :

```
1: 33
2: 22
3: 11
Exception: Stack Empty
```

Answer Question 3

1. code declaration for Levels of inheritance. With the UML illustration:

code declaration	UML
<pre> class A { }; // Base class A class B : public A { }; // Class B derived from A class C : public B { }; // Class C derived from B </pre>	

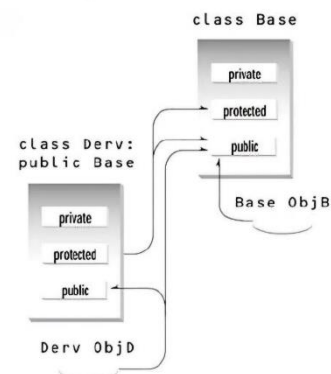
2. types of inheritance with the accessibilities:

Inheritance access types in C++:

- Public inheritance
- Protected inheritance
- Private inheritance

Access Specifier	Accessible from Own Class	Accessible from Derived Class	Accessible from Objects Outside Class
public	yes	yes	yes
protected	yes	yes	no
private	yes	no	no

→ Access Specifiers with inheritance



Answer Question 4

1. code declaration for Multiple inheritance. With the UML illustration.:

code declaration	UML
<pre> class A // base class A { }; class B // base class B { }; class C : public A, public B // C is derived from A and B { }; </pre>	UML class diagram for multiple inheritance.

تابع سؤال 4 فرع 2

2. Illustrate the Class Templates:

- Class templates in C++ enable reusability by defining one generic class and instantiating it for different data types.

Example:

```
#include <iostream>
using namespace std;

template <class T>
class Stack {
private:
    T st[3];
    int top;
public:
    Stack() : top(-1) { }
    void push(T var) { st[++top] = var; }
    T pop() { return st[top--]; }
};

int main() {
    Stack<float> s1;
    s1.push(1111.1);
    s1.push(2222.2);
    s1.push(3333.3);
    cout << s1.pop() << endl;
    cout << s1.pop() << endl;
    cout << s1.pop() << endl;

    Stack<long> s2;
    s2.push(123123123);
    s2.push(234234234);
    s2.push(345345345);
    cout << s2.pop() << endl;
    cout << s2.pop() << endl;
    cout << s2.pop() << endl;

    return 0;
}
```

Output :

```
1: 3333.3
2: 2222.2
3: 1111.1
1: 345345345
2: 234234234
3: 123123123
```

Instantiation:

1. Stack<float> → creates a stack of floats
2. Stack<long> → creates a stack of long integers

Answer Question 5

1. code declaration for template class. With an abstraction of two objects of different data types:

```
template <class T>
class Sample {
private:
    T data;
public:
    Sample(T d) : data(d) {}
};

Sample<int> obj1(100);
Sample<float> obj2(55.5f);
```

2. Illustrate the Static data members :

Definition:

A static data member is one variable shared by all objects of a class.

Example:

```
class Test {
public:
    static int x;
    Test() { x++; }
};

int Test::x = 0;
```

Objects: If we create 3 objects → x = 3 (shared).

